Vision

Multiple inheritance

The goal of this project is to build a framework that implements multiple inheritance for homogeneous class hierarchies in Java. Since Java does not natively support multiple inheritance of state and behavior, this tool will utilize custom annotations and code generation (or instrumentation) to achieve full method combination and dispatching—similar to the Common Lisp Object System.

Key capabilities of the framework:

1. Support the definition of a homogeneous hierarchy, where a single root Java interface defines all operations for the entire hierarchy.
2. Automatically generate an auxiliary class from the annotated root interface, which will act as the actual superclass from the JVM's perspective.
3. Provide a call-next-method mechanism (like in CLOS) by automatically generating a counterpart "next" method for every operation defined in the root interface.
4. Allow developers to declare actual multiple superclasses for any class in the hierarchy using custom class-level annotations.
5. Guarantee proper method dispatching, method combination, and hierarchy traversal using object composition (via instrumentation of the annotated classes or generation of additional proxy classes).
6. Manage object state under the strict assumption that all properties within the hierarchy classes are private.

Current assumptions about the architecture:

1. The framework will heavily rely on compile-time Annotation Processing (APT) or bytecode instrumentation to generate the required auxiliary classes and dispatch logic.
2. From the perspective of the standard Java compiler, all classes in the hierarchy will explicitly inherit only from the single generated auxiliary root class.
3. The multiple inheritance mapping and Method Resolution Order (MRO) will be resolved based on the custom annotations provided by the developer.
4. The dispatch mechanism will use composition under the hood to route method calls through the conceptual hierarchy, seamlessly enabling the "next-method" execution flow across parallel branches of the inheritance tree.
5. The core root must strictly be a Java interface, and clients of the hierarchy will interact with the objects exclusively through this interface type.

Implementation Steps

The implementation uses Java Annotation Processing Tool (APT) for compile-time code generation. We'll generate Java source files for the auxiliary root class and wrapper/delegate classes for composition.

1. Define Custom Annotations

Create annotations to mark the root interface and classes with multiple parents. These will be processed at compile-time.

- **@RootHierarchy**: Applied to the root interface.
- **@ExtendsMultiple**: Applied to classes, specifying parent classes.

```java
@Target(ElementType.TYPE)  no usages
@Retention(RetentionPolicy.SOURCE)
public @interface RootHierarchy {}

@Target(ElementType.TYPE)  no usages
@Retention(RetentionPolicy.SOURCE)
public @interface ExtendsMultiple {
    Class<?>[] parents();  no usages
}
```

2. Implement the Annotation Processor

Create a processor extending AbstractProcessor to handle annotations. It scans for annotated elements and generates code.

```java
@SupportedAnnotationTypes({"RootHierarchy", "ExtendsMultiple"})  no usages
@SupportedSourceVersion(SourceVersion.RELEASE_21)
public class MultipleInheritanceProcessor extends AbstractProcessor {

    @Override
    public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment roundEnv) {
        // Handle @RootHierarchy: generate root class
        roundEnv.getElementsAnnotatedWith(RootHierarchy.class).forEach(e -> generateRootClass((TypeElement) e));

        // Handle @ExtendsMultiple: generate wrappers
        roundEnv.getElementsAnnotatedWith(ExtendsMultiple.class).forEach(e -> generateWrapperClass((TypeElement) e));

        return true;
    }

    // private void generateRootClass(TypeElement iface) {}
    // private void generateWrapperClass(TypeElement cls) {}
}
```

3. Generate the Auxiliary Root Class

For an annotated interface, generate XXXRoot. It implements the interface, adds MRO fields, dispatch logic, and next-methods.

4. Generate Wrapper Classes for Multiple Inheritance

For a class with @ExtendsMultiple, generate a wrapper (e.g., ChildWrapper) that extends the root, initializes delegates, computes MRO, and dispatches.

5. Implement Dispatching and Method Combination

In generated code, use composition for delegation. Track MRO(Calculate MRO using C3) position for next-calls.

6. Example

```
@RootHierarchy  no usages
interface A { void method(); }

class B extends ARoot { void method() { System.out.println("B"); nextMethod(); } }  1 usage

class C extends ARoot { void method() { System.out.println("C"); nextMethod(); } }  1 usage

@ExtendsMultiple(parents = {B.class, C.class})  no usages
class D extends ARoot { void method() { System.out.println("D"); nextMethod(); } }

// Usage: A obj = new DWrapper(); obj.method();
```